Laporan Tugas Besar

IF2224 Teori Bahasa Formal dan Automata

Milestone 1 - Lexical Analysis

Disusun oleh:

13524071 Kalyca Nathania B Manullang

13524073 Keisha Rizka Syofyani

13524087 Muhammad Fakhry Zaki

13524109 Helena Kristela Sarhawa

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Landasan Teori

2.1 Analisis Leksikal dan Peran Lexer

Analisis leksikal (*lexical analysis*) merupakan tahap awal dalam proses kompilasi atau interpretasi suatu program. Pada tahap ini, program sumber yang berupa rangkaian karakter akan diproses menjadi unit-unit yang lebih kecil dan bermakna, yang disebut sebagai *token*. Proses ini dilakukan oleh komponen yang dikenal sebagai *lexical analyzer* atau *lexer*.

Lexer bertugas membaca input secara berurutan dari awal hingga akhir, satu karakter demi satu karakter, dan mengelompokkan karakter-karakter tersebut ke dalam token sesuai dengan aturan bahasa yang digunakan. Selain itu, lexer juga memiliki tanggung jawab untuk mengabaikan karakter yang tidak memiliki makna sintaksis secara langsung, seperti spasi, tab, dan newline, kecuali jika karakter tersebut berperan dalam membedakan token tertentu. Hasil dari proses analisis leksikal ini akan menjadi input bagi tahap selanjutnya, yaitu analisis sintaksis (*parsing*). Oleh karena itu, keakuratan lexer sangat penting karena kesalahan pada tahap ini dapat mempengaruhi keseluruhan proses kompilasi.

2.2 Token, Lexeme, dan Klasifikasi Simbol

Dalam proses analisis leksikal, terdapat konsep dasar yang perlu dipahami, yaitu *token* dan *lexeme*. Token merupakan representasi abstrak dari suatu unit sintaksis, sedangkan lexeme adalah bentuk konkret berupa rangkaian karakter yang diambil dari input. Sebagai contoh, rangkaian karakter **nilai123** merupakan lexeme yang dapat diklasifikasikan sebagai token bertipe *identifier*.

Token dalam suatu bahasa pemrograman umumnya diklasifikasikan ke dalam beberapa kategori, seperti identifier, keyword, operator, konstanta, dan simbol khusus. Identifier digunakan untuk menamai variabel atau entitas lain dalam program dan biasanya terdiri dari kombinasi huruf dan angka. Keyword merupakan kata-kata khusus yang telah didefinisikan dalam bahasa dan memiliki makna tertentu, sehingga tidak dapat digunakan sebagai identifier. Operator mencakup simbol-simbol yang digunakan dalam operasi aritmatika maupun logika, seperti penjumlahan dan perbandingan. Selain itu, terdapat pula konstanta seperti bilangan bulat

(*integer*) dan bilangan riil (*real*), serta string yang merupakan kumpulan karakter. Simbol khusus, seperti tanda kurung, koma, dan titik juga termasuk dalam token karena memiliki peran dalam struktur sintaksis program.

Dalam beberapa bahasa, termasuk bahasa yang digunakan pada tugas ini, keyword bersifat *case-insensitive* yang berarti perbedaan huruf besar dan kecil tidak mempengaruhi makna token. Dengan demikian, penulisan **IF**, **if**, maupun **If** akan dianggap sebagai token yang sama.

2.3 Deterministic Finite Automata (DFA) dalam Lexer

Salah satu konsep utama yang mendasari implementasi lexer adalah *Deterministic Finite Automata* (DFA). DFA merupakan model matematis yang digunakan untuk mengenali pola dalam rangkaian input. DFA terdiri dari sejumlah state, sebuah state awal, satu atau lebih state akhir (final state), serta fungsi transisi yang menentukan perpindahan antar state berdasarkan input yang dibaca.

Dalam konteks analisis leksikal, DFA digunakan untuk memodelkan proses pengenalan token. Setiap jenis token memiliki pola tertentu yang dapat direpresentasikan dalam bentuk state dan transisi. Lexer akan membaca karakter satu per satu dan berpindah dari satu state ke state lain hingga mencapai state akhir yang menandakan bahwa sebuah token telah dikenali.

Salah satu karakteristik penting dari DFA adalah sifat deterministiknya, yaitu pada setiap state dan input tertentu hanya terdapat satu kemungkinan transisi. Selain itu, DFA bekerja secara sekuensial tanpa melakukan pembacaan mundur (*backtracking*) sehingga setiap karakter hanya dibaca sekali dari kiri ke kanan. Hal ini membuat implementasi lexer menjadi efisien dan sesuai dengan prinsip dasar analisis leksikal.

2.4 Pengolahan Konstanta, String, dan Komentar

Dalam analisis leksikal, pengolahan konstanta merupakan salah satu aspek penting. Konstanta numerik umumnya dibagi menjadi bilangan bulat (*integer*) dan bilangan riil (*real*). Bilangan bulat terdiri dari digit tanpa bagian desimal, sedangkan bilangan riil memiliki bagian desimal yang dipisahkan oleh tanda titik. Pada beberapa spesifikasi bahasa, terdapat aturan

tertentu seperti keharusan adanya digit sebelum tanda titik agar suatu bilangan dapat dikenali sebagai bilangan riil.

Selain konstanta numerik, string juga merupakan bagian penting dalam bahasa pemrograman. String biasanya ditandai dengan pembuka dan penutup berupa tanda petik. Dalam beberapa kasus, diperlukan mekanisme *escape character* untuk merepresentasikan karakter khusus di dalam string, misalnya untuk menuliskan tanda petik itu sendiri.

Komentar merupakan bagian dari kode yang digunakan untuk memberikan penjelasan tambahan dan tidak mempengaruhi eksekusi program. Dalam analisis leksikal, komentar harus dikenali dan diabaikan oleh lexer. Komentar dapat memiliki berbagai bentuk seperti yang dibatasi oleh tanda tertentu dan dapat mencakup satu baris maupun beberapa baris. Penanganan komentar harus dilakukan dengan hati-hati, terutama untuk memastikan bahwa batas awal dan akhir komentar dikenali dengan benar.

2.5 Penanganan Error Leksikal dan Mekanisme Input/Output

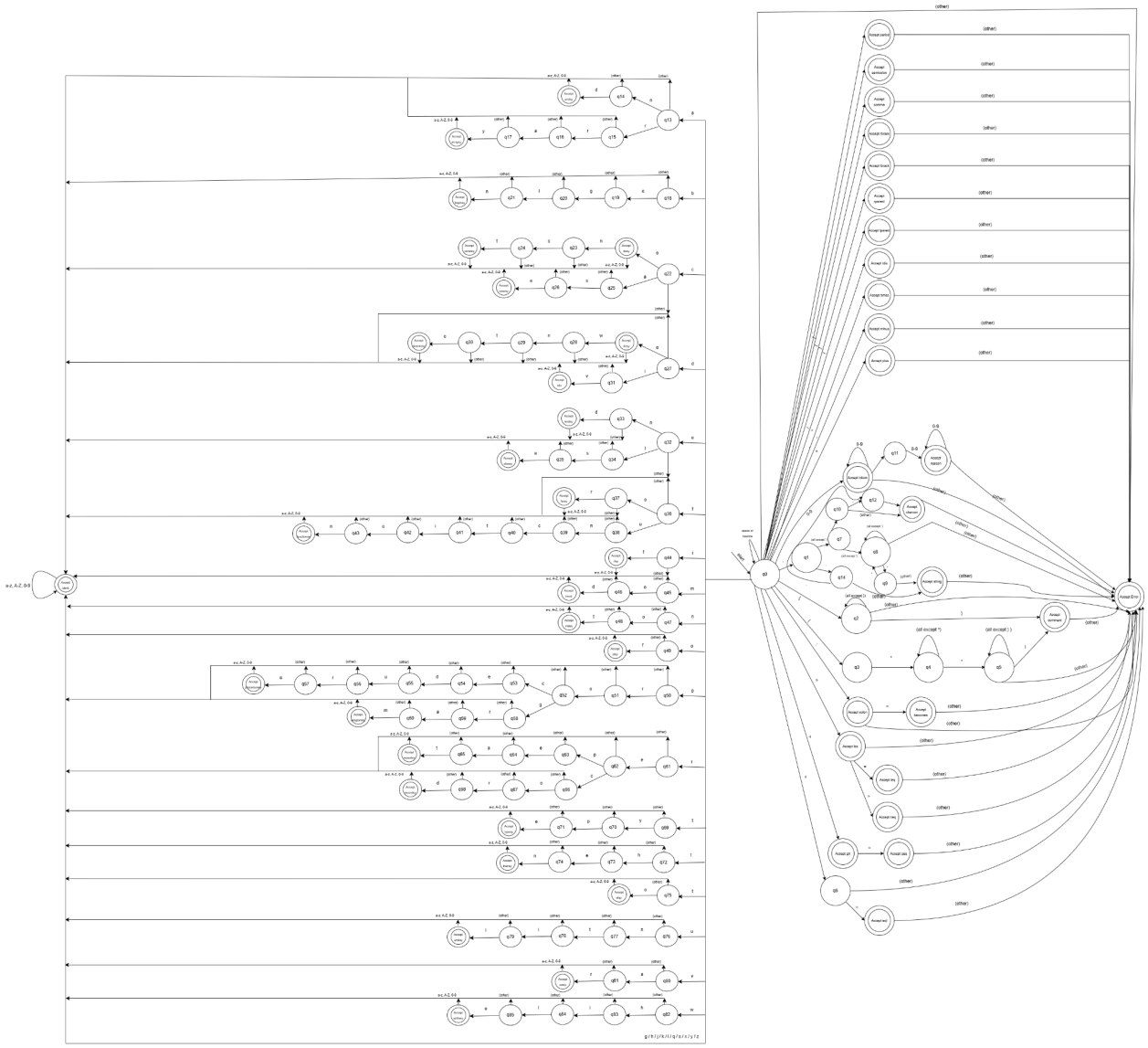
Selama proses analisis leksikal, terdapat kemungkinan munculnya kesalahan yang disebut sebagai *lexical error*. Kesalahan ini terjadi ketika rangkaian karakter dalam input tidak sesuai dengan aturan bahasa yang telah ditentukan. Contoh kesalahan leksikal meliputi karakter yang tidak dikenal, string yang tidak ditutup, atau format angka yang tidak valid.

Dalam implementasi lexer, kesalahan ini umumnya ditandai dengan token khusus, misalnya **unknown** sehingga proses analisis dapat tetap dilanjutkan hingga akhir input. Pendekatan ini memungkinkan sistem untuk melaporkan lebih dari satu kesalahan dalam satu kali proses yang sangat berguna dalam proses debugging.

Proses pembacaan input biasanya dilakukan menggunakan mekanisme *input stream* yang memungkinkan file dibaca secara berurutan karakter demi karakter. Hasil tokenisasi kemudian disimpan dalam struktur data seperti array atau vector dan dapat ditampilkan ke layar atau disimpan kembali ke dalam file menggunakan *output stream*. Mekanisme ini mendukung fleksibilitas dalam pengolahan dan penyimpanan hasil analisis leksikal.

Perancangan & Implementasi

a. Diagram DFA



Gambar 1 Diagram DFA

(source : https://drive.google.com/file/d/1zGQMK9JnMZvJoa61DYVtTVT-w6-2vLHH/view?usp=sharing)

Diagram DFA yang dibuat digunakan untuk mengenali token-token yang diberikan pada spesifikasi tugas dengan membaca input karakter per karakter. DFA ini memiliki beberapa *state* yang mewakili jenis token yang sedang diproses, mulai dari *identifier*, tipe-tipe data primitif, *keyword-keyword* khusus, simbol operator dan *assignment,* serta komentar. Diagram transisi ini terdiri atas *start state* (q0) yang dapat menerima semua

karakter input. Terdapat 85 *state* transisi yang menghubungkan setiap urutan input karakter dan melakukan validasi dengan token yang tersedia dan akan menuju *final state acceptance* jika seluruh karakter yang dibaca sesuai dengan token yang didefinisikan. Apabila urutan karakter yang diterima selama transisi tidak valid sesuai token-token yang tersedia, maka transisi akan masuk ke *final state acceptance error* agar dapat mengembalikan keterangan error.

State q1 dan transisi lanjutannya merepresentasikan token-token tipe data primitif, seperti realcon, intcon, charcon, dan string. Diagram ini juga menunjukkan bagaimana transisi *state* pada penggunaan *escape character* untuk kategori string dan charcon yang didefinisikan sebagai tanda petik satu dua kali (‘’).

Diagram mengelola transisi input karakter yang merujuk pada keyword tertentu yang valid sesuai token yang tersedia, seperti program, downto, end, while, do, until, dan lain sebagainya. Apabila karakter-karakter yang diterima ini tidak terdapat pada kamus token, maka akan diterima sebagai identifier.

Selain itu, diagram juga menunjukkan transisi dalam penerimaan token-token operator. Terdapat beberapa operator yang membutuhkan karakter kedua setelah karakter pertama dibaca, seperti ==, >=, <=, :=, <> yang *output*-nya akan berbeda dengan kondisi ketika hanya karakter pertama yang muncul. Diagram ini pun menangani kondisi ketika operator sama dengan hanya muncul sekali, yang akhirnya akan berujung pada *final state acceptance error*.

Untuk token komentar, diagram menangani dua kasus yang berbeda, yakni penggunaan tanda kurung kurawal ({) dan tanda kurung yang diikuti oleh bintang ((*). Karakter-karakter berikutnya akan dibaca hingga diakhiri oleh tanda penutup yang bersesuaian (} atau *)) sehingga token akan diterima sebagai komentar.

b. Implementasi Program

Dalam mengimplementasikan program untuk membuat lexer, kami membagi *source code* menjadi 3 bagian, yakni main (.cpp dan .hpp), lexer (.cpp dan .hpp), dan dictionary (.hpp). File lexer digunakan untuk menyimpan berbagai fungsi dan prosedur yang berfungsi untuk melakukan lexing dari input file ke output file. File dictionary digunakan untuk menyimpan translasi token/*keyword* dari token input ke token output. Terakhir, file main digunakan untuk

melakukan eksekusi program secara keseluruhan dengan memanfaatkan fungsi-fungsi dan prosedur-prosedur dari lexer.

Pada saat program dijalankan, hal yang pertama dilakukan adalah melakukan inisiasi *dictionary* untuk disimpan ke variabel global bernama *dictionary* agar bisa digunakan oleh lexer kedepannya. Setelah melakukan inisiasi *dictionary*, program akan meminta user untuk meng-*input* nama file yang ingin dilakukan lexing. Program kemudian akan membaca file tersebut karakter demi karakter hingga sampai ke akhir file (EOF). Pada saat melakukan pembacaan karakter, program terbagi menjadi beberapa *state* tergantung karakter atau karakteristik dari karakter yang sedang dibaca. *State* program terbagi menjadi *state* membaca *blank* (spasi atau *endline*), *state* membaca angka, *state* membaca *keyword* atau *identifier*, dan *state* membaca karakter khusus.

Pada *state* pembacaan karakter kosong, program akan melakukan pembiaran dan langsung lanjut membaca karakter berikutnya hingga menemukan karakter yang bukan karakter kosong. Pada *state* pembacaan angka, program akan melanjutkan pembacaan hingga membaca karakter yang bukan termasuk angka. Kemudian akumulasi dari kumpulan karakter yang telah dibaca tersebut digabungkan menjadi satu kesatuan intcon atau realcon (sesuai angka yang dibaca merupakan desimal atau bukan). Pada *state* pembacaan *keyword* atau *identifier*, program akan melanjutkan membaca hingga menemukan karakter yang membuatnya keluar dari state ini, yakni karakter-karakter khusus atau karakter kosong. Sama seperti pada pembacaan angka, akumulasi dari kumpulan karakter yang telah dibaca digabungkan menjadi satu kesatuan. Apabila kumpulan karakter tersebut merupakan sebuah token yang valid (terdapat dalam *dictionary*), kumpulan karakter tersebut akan ditranslasikan sesuai *mapping keyword*-nya pada *dictionary*. Sedangkan apabila kumpulan karakter tersebut bukan merupakan token yang valid, kumpulan karakter tersebut akan dianggap sebagai *identifier*. Terakhir, pada *state* pembacaan karakter khusus, program juga akan melakukan pembacaan hingga akhir karakter khusus. Selanjutnya, apabila kumpulan karakter khusus tersebut valid (terdapat dalam *dictionary*), kumpulan karakter tersebut akan ditranslasikan sesuai *mapping keyword*-nya pada *dictionary*. Sedangkan, apabila kumpulan karakter khusus tersebut tidak valid, kumpulan karakter akan dianggap sebagai *unknown*.

Setelah program selesai membaca file input (mencapai EOF), program akan menampilkan hasil lexing pada terminal dan akan meminta user untuk memasukkan nama file

untuk menyimpan file output. Terakhir, hasil file yang sudah di-lexing tersebut tersimpan dalam file dengan nama sesuai dengan yang dimasukan user.

Pengujian

1. Test Case 1

Pengujian kasus sederhana secara keseluruhan sesuai pada file input sesuai contoh pada dokumen spesifikasi.

<table>
<tr><td>Input</td><td>

```
test > milestone-1 > input-1.txt
1  program Test1;
2
3  var
4    a, b: integer;
5
6  begin
7    a := 5;
8    b := a + 10;
9  end.
```

</td></tr>
<tr><td>Proses</td><td>

```
Masukkan nama file input : input-1.txt
programsy
ident (Test1)
semicolon
varsy
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon
beginsy
ident (a)
becomes
intcon (5)
semicolon
ident (b)
becomes
ident (a)
plus
intcon (10)
semicolon
endsy
period
Masukkan nama file untuk menyimpan hasil : output-1
```

</td></tr>
</table>

Output	test > milestone-1 > output-1.txt programsy ident (Test1) semicolon varsy ident (a) comma ident (b) colon ident (integer) semicolon beginsy ident (a) becomes intcon (5) semicolon ident (b) becomes ident (a) plus intcon (10) semicolon endsy period

2. Test Case 2

Pengujian kasus pada penggunaan simbol-simbol operator

Input	test > milestone-1 > input-2.txt
	`program OpsTest;`
	
	`var`
	`  x, y: integer;`
	
	`begin`
	`  x := 10;`
	`  y := 5;`
	
	`  if x == y then`
	`  x := x + y;`
	`  if x <> y then`
	`  x := x - y;`
	`  if x > y then`
	`  x := x * y;`
	`  if x >= y then`
	`  x := x div y;`
	`  if x < y then`
	`  x := x / y;`
	`  if x <= y then`
	`  x := x MOD y;`
	
	`  if NOT (x == y) AND (x > y) OR (y < x) then`
	`  x := y;`
	`end.`

Proses	`Masukkan nama file input : input-2.txt`
	`programsy`
	`ident (OpsTest)`
	`semicolon`
	`varsy`
	`ident (x)`
	`comma`
	`ident (y)`
	`colon`
	`ident (integer)`
	`semicolon`
	`beginsy`
	`ident (x)`
	`becomes`
	`intcon (10)`
	`semicolon`
	`gtr`
	`ident (y)`
	`rparent`
	`orsy`
	`lparent`
	`ident (y)`
	`lss`
	`ident (x)`
	`rparent`
	`thensy`
	`ident (x)`
	`becomes`
	`ident (y)`
	`semicolon`
	`endsy`
	`period`
	`Masukkan nama file untuk menyimpan hasil : output-2`

Output

test > milestone-1 > output-2.txt

```
programsy
ident (OpsTest)
semicolon
varsy
ident (x)
comma
ident (y)
colon
ident (integer)
semicolon
beginsy
ident (x)
becomes
intcon (10)
semicolon
ident (y)
becomes
intcon (5)
semicolon
ifsy
ident (x)
eql
ident (y)
thensy
ident (x)
becomes
ident (x)
plus
ident (y)
semicolon
ifsy
ident (x)
neq
ident (y)
thensy
ident (x)
becomes
ident (x)
minus
ident (y)
semicolon
ifsy
ident (x)
gtr
ident (y)
thensy
ident (x)
becomes
ident (x)
times
ident (y)
semicolon
ifsy
ident (x)
geq
ident (y)
thensy
ident (x)
becomes
ident (x)
idiv
```

test > milestone-1 > output-2.txt

```
idiv
ident (y)
semicolon
ifsy
ident (x)
lss
ident (y)
thensy
ident (x)
becomes
ident (x)
rdiv
ident (y)
semicolon
ifsy
ident (x)
leq
ident (y)
thensy
ident (x)
becomes
ident (x)
imod
ident (y)
semicolon
ifsy
notsy
lparent
ident (x)
eql
ident (y)
rparent
andsy
lparent
ident (x)
gtr
ident (y)
rparent
orsy
lparent
ident (y)
lss
ident (x)
rparent
thensy
ident (x)
becomes
ident (y)
semicolon
endsy
period
```

3. Test Case 3

Pengujian untuk kasus tipe-tipe data primitif *character* dan string beserta penggunaan *escape character*-nya. Pada kasus ini juga diuji bagaimana lexer menangani kasus tidak valid.

Input

test > milestone-1 > input-3.txt

```
program DataTest;

type
  myArray = array[1..10] of integer;
  myRecord = record
    c: char;
    s: string;
  end;

var
  a: myArray;
  r: myRecord;

begin
  r.c := 'A';
  r.s := 'HelloWorld';

  { ===== TEST ESCAPE CHARACTER ===== }

  r.s := 'It''s fine';          { escape di tengah }
  r.s := 'I''m learning';       { escape di tengah }
  r.s := '''';                  { 1 karakter ' }
  r.s := 'A''B''C';             { multiple escape }
  r.s := '123''456';            { angka + escape }

  { ===== EDGE CASE ===== }

  r.s := '';                    { string kosong }
  r.s := ' ';                   { spasi }
  r.s := '''Hello''';           { quote di awal & akhir }
  r.s := 'End with quote''';    { escape di akhir }

  { ===== ERROR CASE ===== }

  r.s := 'Unclosed string;      { tidak ditutup }
  r.s := 'Line
break';                         { multiline (error) }

end.
```

Proses	``` Masukkan nama file input : input-3.txt Lexical error: string tidak boleh multiline Lexical error: string tidak boleh multiline Lexical error: string tidak boleh multiline programsy ident (DataTest) semicolon typesy ident (myArray) unknown (=) arraysy lbrack intcon (1) period period intcon (10) ``` ``` string (End with quote') semicolon ident (r) period ident (s) becomes unknown (Unclosed string; { tidak ditutup }) ident (r) period ident (s) becomes unknown (Line) ident (break) unknown (; { multiline (error) }) endsy period Masukkan nama file untuk menyimpan hasil : output-3 ``` ``` ```

Output	test > milestone-1 > output-3.txt programsy ident (DataTest) semicolon typesy ident (myArray) unknown (=) arraysy lbrack intcon (1) period period intcon (10) rbrack ofsy ident (integer) semicolon ident (myRecord) unknown (=) recordsy ident (c) colon ident (char) semicolon ident (s) colon ident (string) semicolon endsy semicolon varsy ident (a) colon ident (myArray) semicolon ident (r) colon ident (myRecord) semicolon beginsy ident (r) period

```
ident (c)
becomes
charcon (A)
semicolon
ident (r)
period
ident (s)
becomes
string (HelloWorld)
semicolon
ident (r)
period
ident (s)
becomes
string (It's fine)
semicolon
ident (r)
period
ident (s)
becomes
string (I'm learning)
semicolon
ident (r)
period
ident (s)
becomes
charcon (')
semicolon
ident (r)
period
ident (s)
becomes
string (A'B'C)
semicolon
ident (r)
period
ident (s)
becomes
string (123'456)
semicolon
ident (r)
```

```
period
ident (s)
becomes
string
semicolon
ident (r)
period
ident (s)
becomes
charcon ( )
semicolon
ident (r)
period
ident (s)
becomes
string ('Hello')
semicolon
ident (r)
period
ident (s)
becomes
string (End with quote')
semicolon
ident (r)
period
ident (s)
becomes
unknown (Unclosed string;      { tidak ditutup })
ident (r)
period
ident (s)
becomes
unknown (Line)
ident (break)
unknown (;                        { multiline (error) })
endsy
period
```

4. Test Case 4

Pengujian pada penggunaan *keyword-keyword* khusus yang sudah terdefinisi sesuai daftar token yang tersedia.

Input	test > milestone-1 > input-4.txt
	`program LoopTest;`
	
	`var`
	`  i: integer;`
	
	`begin`
	`  for i := 1 to 10 do`
	`  i := i + 1;`
	
	`  for i := 10 downto 1 do`
	`  i := i - 1;`
	
	`  while i > 0 do`
	`  i := i - 1;`
	
	`  repeat`
	`  i := i + 1;`
	`  until i == 10;`
	
	`  case i of`
	`  1: i := 100;`
	`  2: i := 200;`
	`  end;`
	`end.`

Proses	`Masukkan nama file input : input-4.txt`
	`programsy`
	`ident (LoopTest)`
	`semicolon`
	`varsy`
	`ident (i)`
	`colon`
	`ident (integer)`
	`semicolon`
	`beginsy`
	`forsy`
	`ident (i)`
	`becomes`
	`intcon (1)`
	`tosy`
	
	`colon`
	`ident (i)`
	`becomes`
	`intcon (100)`
	`semicolon`
	`intcon (2)`
	`colon`
	`ident (i)`
	`becomes`
	`intcon (200)`
	`semicolon`
	`endsy`
	`semicolon`
	`endsy`
	`period`
	`Masukkan nama file untuk menyimpan hasil : output-4`

Output	test > milestone-1 > output-4.txt programsy ident (LoopTest) semicolon varsy ident (i) colon ident (integer) semicolon beginsy forsy ident (i) becomes intcon (1) tosy intcon (10) dosy ident (i) becomes ident (i) plus intcon (1) semicolon forsy ident (i) becomes intcon (10) downtosy intcon (1) dosy ident (i) becomes ident (i) minus intcon (1) semicolon whilesy ident (i) gtr intcon (0) dosy ident (i)

```
becomes
ident (i)
minus
intcon (1)
semicolon
repeatsy
ident (i)
becomes
ident (i)
plus
intcon (1)
semicolon
untilsy
ident (i)
eql
intcon (10)
semicolon
casesy
ident (i)
ofsy
intcon (1)
colon
ident (i)
becomes
intcon (100)
semicolon
intcon (2)
colon
ident (i)
becomes
intcon (200)
semicolon
endsy
semicolon
endsy
period
```

5. Test Case 5

Pengujian pada kasus penggunaan token komentar serta kasus penggunaan karakter khusus di dalamnya.

Input	test > milestone-1 > input-5.txt
	``
	program CommentTest;
	
	var
	  x: integer;
	
	begin
	  { ini komentar biasa }
	  x := 5;
	
	  (* ini komentar
	  multi-line *)
	
	  x := x + 10;
	
	  { komentar dengan simbol aneh + - * / := }
	
	  x := 'Not a { comment }';
	end.
	``
	Masukkan nama file input : input-5.txt
	programsy
	ident (CommentTest)
	semicolon
	varsy
	ident (x)
	colon
	ident (integer)
	semicolon
	beginsy
	ident (x)
	becomes
	intcon (5)
	semicolon
	ident (x)
	becomes
	ident (x)
	plus
	intcon (10)
	semicolon
	ident (x)
	becomes
	string (Not a { comment })
	semicolon
	endsy
	period
	Masukkan nama file untuk menyimpan hasil : output-5

Proses	

Output	test > milestone-1 > output-5.txt programsy ident (CommentTest) semicolon varsy ident (x) colon ident (integer) semicolon beginsy ident (x) becomes intcon (5) semicolon ident (x) becomes ident (x) plus intcon (10) semicolon ident (x) becomes string (Not a { comment }) semicolon endsy period

Kesimpulan dan Saran

Kesimpulan

Berdasarkan hasil yang telah didapat, program lexer berhasil dibuat untuk membaca karakter secara sekuensial dan mengelompokkan serta membagi karakter-karakter yang dibaca menjadi token-token. Selain berhasil diimplementasikan dalam program, hasil menunjukkan bahwa proses lexing bisa dilakukan dengan diagram transisi DFA. Dengan membaca berbagai karakter, program berpindah dari suatu *state* ke *state* yang lain hingga mencapai final *state*. Dengan transisi ini pula, beberapa error leksikal bisa ditangani seperti pembacaan token/*keyword* yang tidak dikenal.

Saran

Karena tugas pada milestone 1 ini hanya terbatas pada implementasi lexer saja, tentunya masih banyak komponen penting dalam *compiler* yang belum diimplementasikan. Agar alur pembuatan program dan batasan tiap milestone lebih terbayang, sebaiknya di awal diberikan gambaran lebih jelas mengenai tugas besar secara keseluruhan dan pembagian capaian tiap milestone-nya.

Lampiran

1. Link Repository Github
   https://github.com/keirzka/AMN-Tubes-IF2224-2026.git

2. Link Workspace Diagram Transisi DFA
   https://drive.google.com/file/d/1zGQMK9JnMZvJoa61DYVtTVT-w6-2vLHH/view?usp=sharing

3. Pembagian Tugas

Nama / NIM	Deskripsi Tugas	Persentase Kontribusi
13524071 Kalyca Nathania Benedicta Manullang	Mengerjakan laporan	25%
	Mengerjakan implementasi program lexer	
13524073 Keisha Rizka Syofyani	Mengerjakan laporan	25%
	Membuat diagram transisi DFA	
	Mengerjakan implementasi program lexer	
13524087 Muhammad Fakhry Zaki	Mengerjakan laporan	25%
	Mengerjakan implementasi program lexer	
	Membuat dictionary	
13524109 Helena Kristela Sarhawa	Membuat diagram transisi DFA	25%
	Mengerjakan implementasi program lexer	
	Mengerjakan laporan	